FACULTY OF COMPUTING

UNIVERSITI TEKNOLOGI MALAYSIA

DATA STRUCTURE AND ALGORITHM

SECJ2013-02

SEMESTER 1 2024/2025

MINI PROJECT: FLIGHT PASSENGER QUEUE SIMULATION AT AIRPORT

LECTURER: DR ZURAINI BINTI ALI SHAH

GROUP 5

Name	Matric No.
BRENDAN CHIA YAN FEI (LEADER)	A23CS0211
GOH JIALE	A22EA0043
SALMAN SUHAIB MAJEED MAJEED	A22EC4013
YASMIN BATRISYIA BINTI ZAHIRUDDIN	A23CS0201

Table of Contents

1. Introduction

1.1 Synopsis of Project

Our project title is “Flight Passenger Queue Simulation at Airport”. The project’s main goal is to use C++ to develop simulation of an airport’s flight passenger queue system. The system will demonstrate the use of queues, to control passenger flow during arrivals and departure. Besides, the system efficiently organizes passengers into a queue based on their priority (Regular, Business, or VIP), simulates their arrival at the check-in counters, manages boarding, and provides real-time updates on the status of the queue. The project focuses on applying data structures and algorithm to solve real-world problems related to queue management in an airport setting

1.2 Project Objectives

1. To model an airport passenger queue system.
2. To build an interactive C++ application with options to simulate passenger arrival, handle departures, and display queue status.
3. To develop a passenger management system
4. To provide features for monitoring queue status (empty, full, current size) and visualising the order of passengers in the queue.
5. To apply data structures and algorithms in solving real-world problems, which is airport queue management.

2. System Requirement, Analysis & Design

2.1 System Requirements (Use Case Diagram)

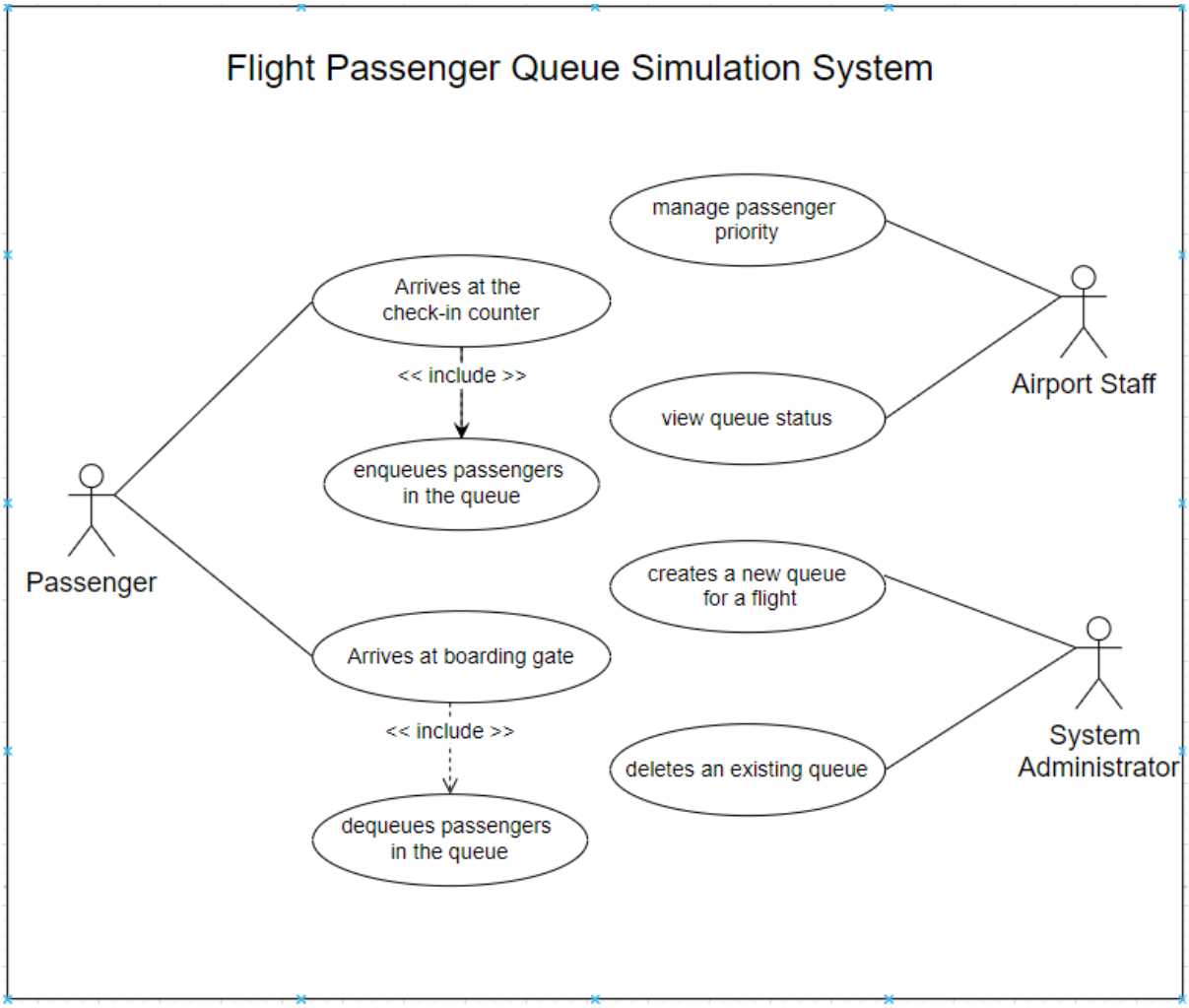


Diagram 2.1 : Use Case Diagram

2.1.1 User Description

User	Description
Passenger	1. Any individual who is travelling by air 2. Categorized into: a. Regular (Standard traveller) b. Business (Individual who paid for upgraded services) c. VIP (Well-known individuals or individuals with particular travel requirements) 3. Different levels of technical ability and familiarity with technology and online platforms.
Airport Staff	1. Employees who are responsible for managing passenger flow and queue operations
System administrator	1. Ensures that the system operates efficiently 2. Handles administrative tasks such as creating, deleting and monitoring queues for flights.

2.1.2 Use Case Description

Use case	Function	Description
UC01	Arrive at check-in counter	Passengers arrive at the check-in counter to register for their flight and are added to the queue based on their priority level.
UC02	Enqueues passengers in the queue	This use case handles the process of inserting passengers into the flight's queue.
UC03	Arrive at the boarding gate	Passengers are processed at the boarding gate based on their priority or position in the queue.
UC04	Dequeues passengers in the queue	This use case manages the removal of passengers from the queue, when passengers is about to board the plane
UC05	Manage passenger priority	Passengers are managed based on their priority levels: Regular, Business, or VIP.
UC06	View queue status	Airport staffs view the current status of the queue
UC07	Create a new queue for a flight	System administrator create s a new queue with a defined capacity
UC08	Deletes an existing queue	This use case clears and deletes a queue after a certain flight's passengers have been processed.

2.2 System Analysis

2.2.1. Problem Description

Nowadays, people at the airport often do not take lining up at checkpoints seriously during departure and arrival.. There are tons of reason why people do not queue. One of the reasons is some people feel like airport has a very stressful environment. This is because, airports are very tight with flight schedules, and long lines. Therefore, it increases the passenger sense of urgency, and they will try to skip the queue, so that they don't miss their flights.

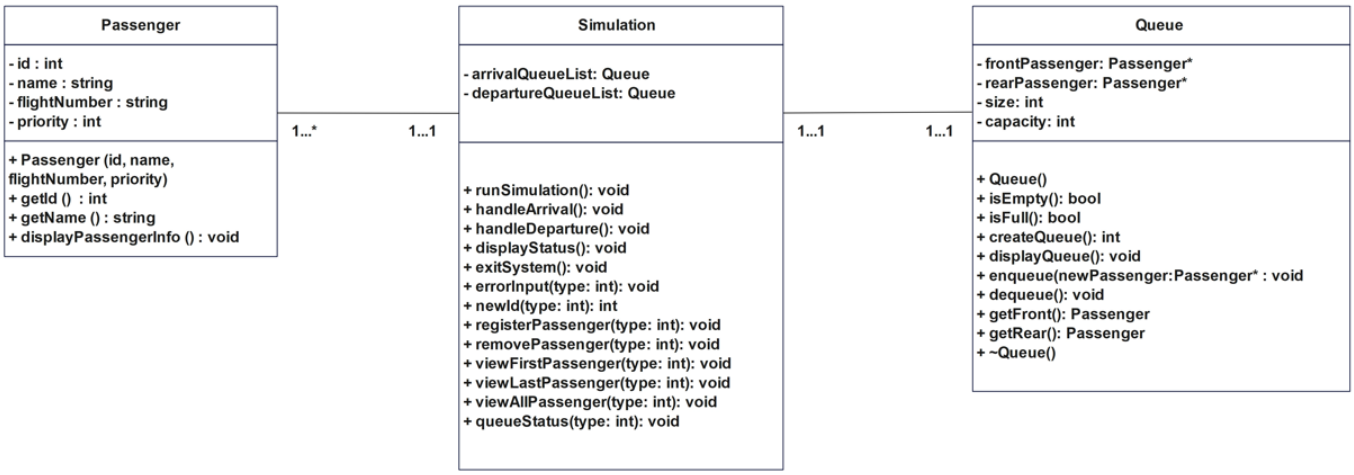
By simulating the arrival, departure and queue system, it will encourage people to follow their turns. Everyone will be treated accordingly by adhering to the First Come, First Serve principle. This principle shows fairness, and easy to understand by people. Theoretically, this principle is equivalent to the Abstract Data Type Queue which apply the concept of First In First Out (FIFO) principle.

2.2.2. Reason queue (FIFO) is suitable for the simulation

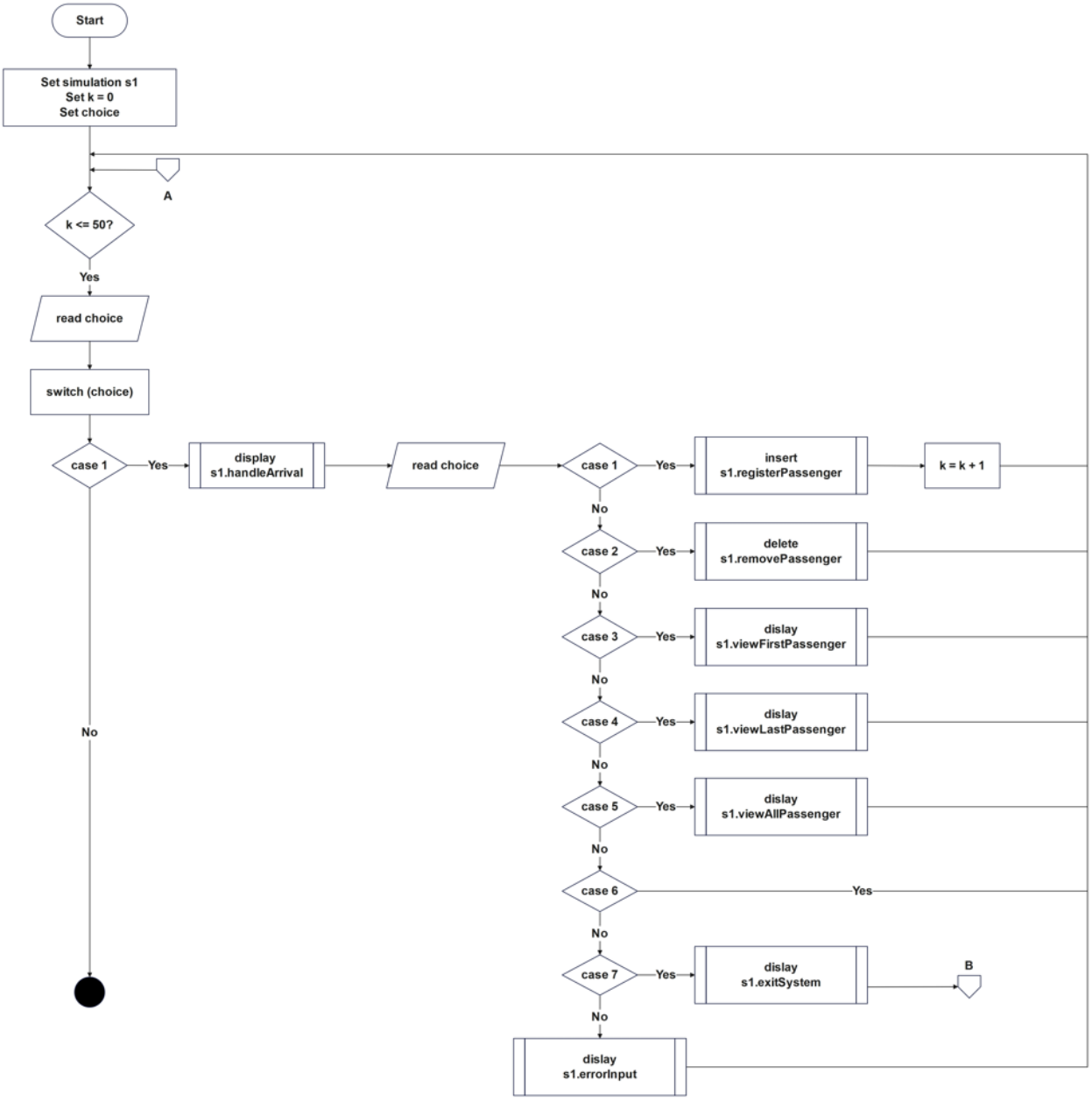
By applying the First In, First Out (FIFO) principle, this can ensure the fairness among the passengers. For example, since the first passenger has been in line for a very long time, they will be served before the rest. Then, It will be out first because of the first passenger matters that have been completed. And the process will go on until the queue maximum size is reached.

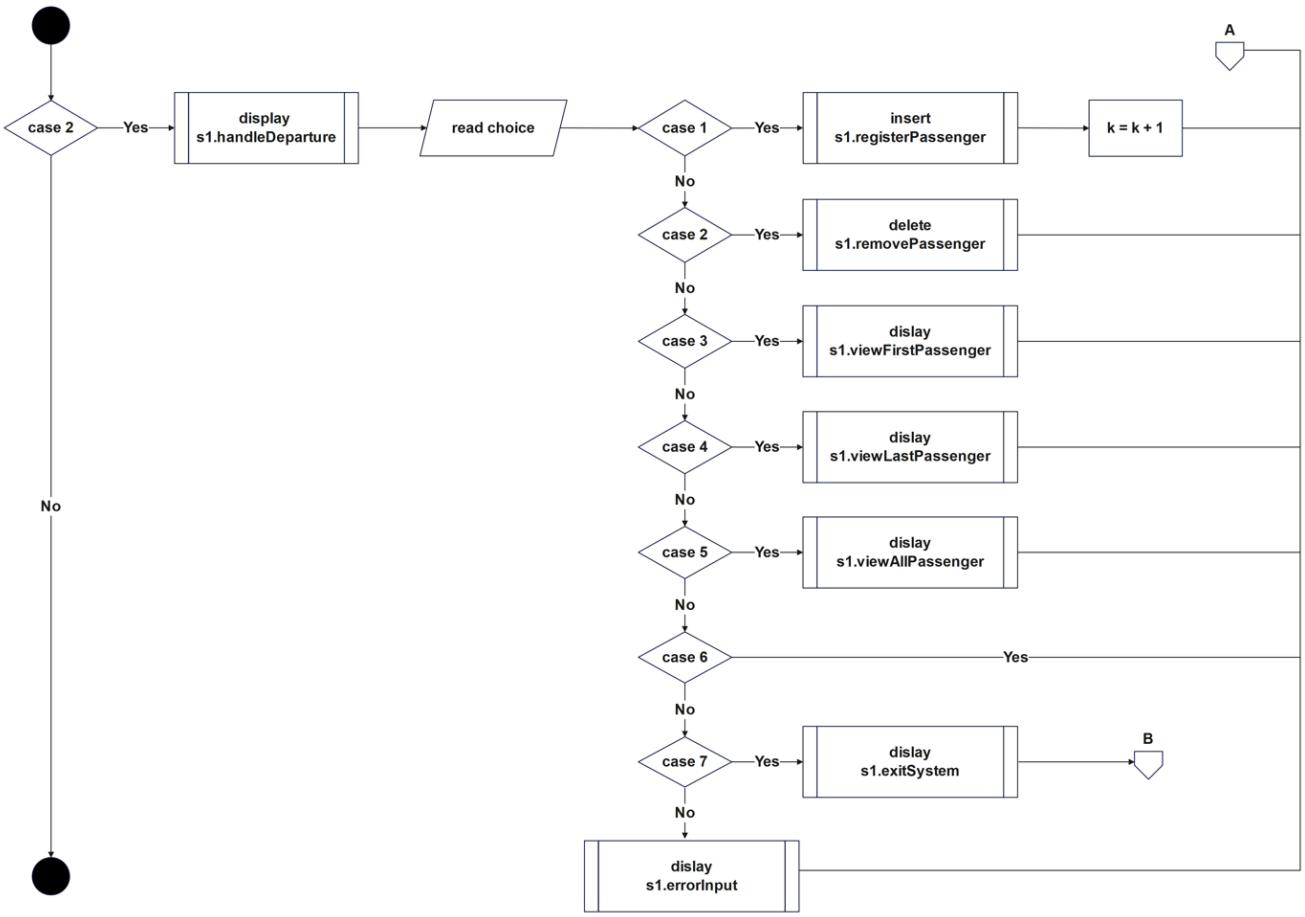
2.3 System Design

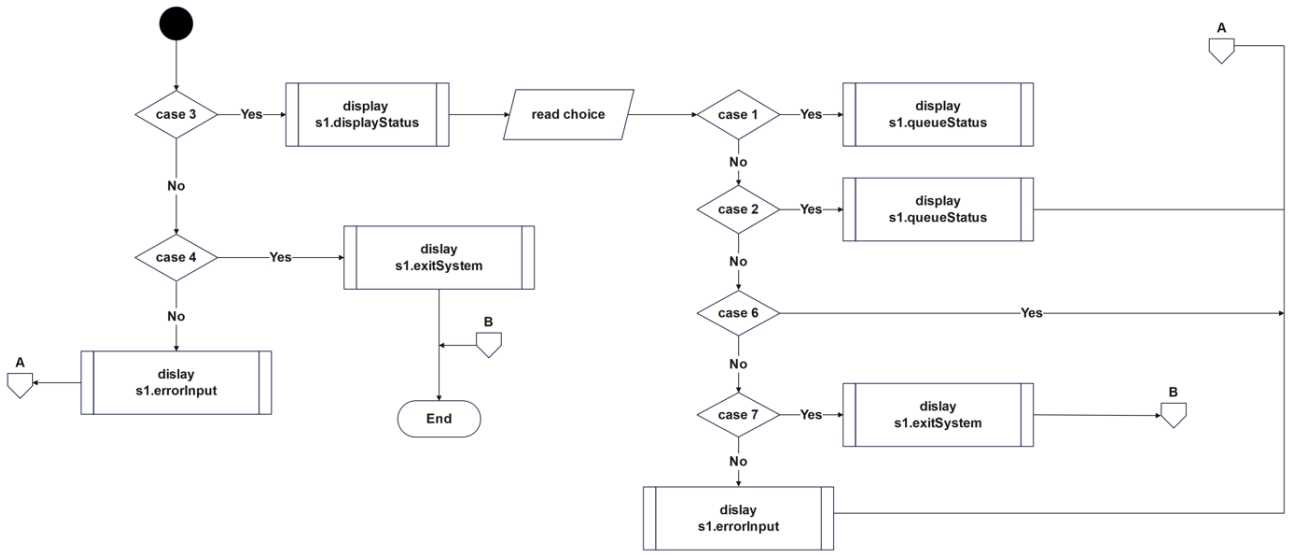
2.3.1 UML Diagram


Passenger
- id : int
- name : string
- flightNumber : string
- priority : int
+ Passenger (id, name, flightNumber, priority)
+ getId () : int
+ getName () : string
+ displayPassengerInfo () : void
1...*
1...1
Simulation
- arrivalQueueList: Queue
- departureQueueList: Queue
+ runSimulation(): void
+ handleArrival(): void
+ handleDeparture(): void
+ displayStatus(): void
+ exitSystem(): void
+ errorInput(type: int): void
+ newId(type: int): int
+ registerPassenger(type: int): void
+ removePassenger(type: int): void
+ viewFirstPassenger(type: int): void
+ viewLastPassenger(type: int): void
+ viewAllPassenger(type: int): void
+ queueStatus(type: int): void
1...1
1...1
Queue
- frontPassenger: Passenger*
- rearPassenger: Passenger*
- size: int
- capacity: int
+ Queue()
+ isEmpty(): bool
+ isFull(): bool
+ createQueue(): int
+ displayQueue(): void
+ enqueue(newPassenger:Passenger* : void
+ dequeue(): void
+ getFront(): Passenger
+ getRear(): Passenger
+ ~Queue()

2.3.2 Flowchart with description for each use case




A
case 2
Yes
display
s1.handleDeparture
read choice
case 1
Yes
insert
s1.registerPassenger
k = k + 1
No
case 2
Yes
delete
s1.removePassenger
No
case 3
Yes
dislay
s1.viewFirstPassenger
No
case 4
Yes
dislay
s1.viewLastPassenger
No
case 5
Yes
dislay
s1.viewAllPassenger
No
case 6
Yes
No
case 7
Yes
dislay
s1.exitSystem
B
No
dislay
s1.errorInput
No


case 3
Yes
display
s1.displayStatus
read choice
case 1
Yes
display
s1.queueStatus
No
case 2
Yes
display
s1.queueStatus
No
case 6
Yes
No
case 7
Yes
dislay
s1.exitSystem
B
No
dislay
s1.errorInput
A
No
case 4
Yes
dislay
s1.exitSystem
B
No
dislay
s1.errorInput
A
End

3. Overall coding in C++.

```cpp
#include <iostream>
using namespace std;

class Passenger{
  private:
    int id;
    string name;
    string flightNumber;
    int priority;           // 1 - Regular, 2 - Business, 3 - VIP
  public:
    Passenger *next;
    Passenger(int id, string name, string flightNumber, int priority){
      this->id = id;
      this->name = name;
      this->flightNumber = flightNumber;
      this->priority = priority;
      next = NULL;
    }
    int getId(){
      return id;
    }
    string getName(){
      return name;
    }
    void DisplayPassengerInfo(){
      cout << "ID             : " << id << endl;
      cout << "Name           : " << name << endl;
      cout << "Flight Number  : " << flightNumber << endl;
      switch (priority){
        case 1:
          cout << "Priority       : Regular" << endl;
          break;

        case 2:
          cout << "Priority       : Business" << endl;
          break;

        case 3:
          cout << "Priority       : VIP" << endl;
          break;

        default:
          break;
      }
    }
```

```
};

class Queue{
    private:
      Passenger *frontPassenger;
      Passenger *rearPassenger;
      int size;
      int capacity;

    public:
      Queue(){
        frontPassenger = nullptr;
        rearPassenger = nullptr;
        size = 0;
        capacity = 50;

      }

      bool isEmpty(){
        return frontPassenger == nullptr;
      }

      bool isFull(){
        return size == capacity;
      }

      int createQueue(){
        int numPassengers;
        cout << "Enter the number of passengers: ";
        cin >> numPassengers;
        return numPassengers;
      }

      void displayQueue(){
        if(isEmpty()){
          cout << "Queue is empty." << endl;
          return;
        }
        Passenger *temp = frontPassenger;
        while(temp!= nullptr){
          temp->DisplayPassengerInfo();
          cout << endl;
          temp = temp->next;
        }
      }
```

```
void enqueue(Passenger *newPassenger){     //insert into queue
  if(isFull()){
    cout << "Queue is full. Cannot enqueue." << endl;
    return;
  }
  if(isEmpty()){
    frontPassenger = newPassenger;
  } else {
    rearPassenger->next = newPassenger;
  }
  rearPassenger = newPassenger;
  size++;
}

void dequeue(){       //delete from
  if(isEmpty()){
    cout << "Queue is empty. Cannot dequeue." << endl;
    return;
  }
  Passenger *temp = frontPassenger;
  frontPassenger = frontPassenger->next;
  delete temp;
  size--;

  if(isEmpty()){
    rearPassenger = nullptr;
  }
}



Passenger getFront(){
  if(isEmpty()){
    cout << "Queue is empty. Cannot get front passenger." << endl;
    return Passenger(0, "", "", 0);
  } else {
    return *frontPassenger;
  }
}

Passenger getRear(){
  if(isEmpty()){
    cout << "Queue is empty. Cannot get rear passenger." << endl;
    return Passenger(0, "", "", 0);
  } else {
    return *rearPassenger;
  }
}
```

```
      ~Queue(){
        while(!isEmpty()){
          getFront();
        }
      }
};

class Simulation{
  private:
    Queue arrivalQueueList;
    Queue departureQueueList;
  public:
    void runSimulation(){
      int choice;

      cout << endl;
      cout << "===== G5 Flight Passenger Queue Management System =====" <<
endl;
      cout << "1. Manage Passenger Arrival" << endl;
      cout << "2. Manage Passenger Departure" << endl;
      cout << "3. View Current Queue Status" << endl;
      cout << "4. Exit System" << endl;
      cout << "=====================================================" <<
endl;
      cout << "Choose an action (from 1 to 4): ";
      cin >> choice;
      cin.ignore();
      cout << endl;

      switch (choice)
      {
      case 1:
        handleArrival();
        break;

      case 2:
        handleDeparture();
        break;

      case 3:
        displayStatus();
        break;

      case 4:
        exitSystem();
```

```
        break;

      default:
        errorInput(0); // 0 indicates back to simulation menu.
        break;
      }
    }

    void handleArrival(){
      int choice = 0;
      cout << "==================== Arrival Flights ==================" <<
endl;
      cout << "1. Register Passenger Arrival" << endl;
      cout << "2. Remove Passenger from Queue" << endl;
      cout << "3. View First Passenger in Queue" << endl;
      cout << "4. View Last Passenger in Queue" << endl;
      cout << "5. View All Passengers in Queue" << endl;
      cout << "6. Go Back" << endl;
      cout << "7. Exit System" << endl;
      cout << "=======================================================" <<
endl;
      cout << "Choose an action (from 1 to 4): ";
      cin >> choice;
      cin.ignore();
      cout << endl;

      switch (choice){
        case 1:
          registerPassenger(0); // 0 indicates arrival type
          break;

        case 2:
          removePassenger(0); // 0 indicates arrival type
          break;

        case 3:
          viewFirstPassenger(0); // 0 indicates arrival type
          break;

        case 4:
          viewLastPassenger(0); // 0 indicates arrival type
          break;

        case 5:
          viewAllPassenger(0); // 0 indicates arrival type
          break;
```

```
        case 6:
          runSimulation();
          break;

        case 7:
          exitSystem();
          break;

        default:
          errorInput(1); // 1 indicates back to manage arrival.
          break;
      }
      runSimulation();
    }

    void handleDeparture(){
      int choice = 0;
      cout << "================= Departure Flights ==================" <<
endl;
      cout << "1. Register Passenger Departure" << endl;
      cout << "2. Remove Passenger from Queue" << endl;
      cout << "3. View First Passenger in Queue" << endl;
      cout << "4. View Last Passenger in Queue" << endl;
      cout << "5. View All Passengers in Queue" << endl;
      cout << "6. Go Back" << endl;
      cout << "7. Exit System" << endl;
      cout << "======================================================" <<
endl;
      cout << "Choose an action (from 1 to 4): ";
      cin >> choice;
      cin.ignore();
      cout << endl;

      switch (choice){
        case 1:
          registerPassenger(1); // 1 indicates departure type
          break;

        case 2:
          removePassenger(1);  // 1 indicates departure type
          break;

        case 3:
          viewFirstPassenger(1);  // 1 indicates departure type
          break;
```

```
        case 4:
          viewLastPassenger(1);  // 1 indicates departure type
          break;

        case 5:
          viewAllPassenger(1);  // 1 indicates departure type
          break;

        case 6:
          runSimulation();
          break;

        case 7:
          exitSystem();
          break;

        default:
          errorInput(2); // 2 indicates back to manage departure.
          break;
      }
      runSimulation();
    }

    void displayStatus(){
      int choice = 0;
      cout << "============== View Current Queue Status ==============" <<
endl;
      cout << "1. Arrival Queue Status" << endl;
      cout << "2. Departure Queue Status" << endl;
      cout << "3. Go Back" << endl;
      cout << "4. Exit System" << endl;
      cout << "=======================================================" <<
endl;
      cout << "Choose an action (from 1 to 4): ";
      cin >> choice;
      cin.ignore();
      cout << endl;

      switch (choice){
        case 1:
          queueStatus(0); // 1 indicates departure type
          break;

        case 2:
          queueStatus(1);  // 1 indicates departure type
```

```
          break;

        case 3:
          runSimulation();
          break;

        case 4:
          exitSystem();
          break;

        default:
          errorInput(3); // 3 indicates back to display status.
          break;
      }
      runSimulation();
    }

    void exitSystem(){
      cout << "Exiting the system. Thank you for using the G5 Flight Passenger
Queue Management System!" << endl;
      exit(0);
    }

    void errorInput(int type){
      cout << "Invalid input. Please enter a number corresponding to the
options provided." << endl;
      switch (type){
        case 0:
          runSimulation();
          break;

        case 1:
          handleArrival();
          break;

        case 2:
          handleDeparture();
          break;

        case 3:
          displayStatus();
          break;

        default:
          break;
      }
```

```
    }

    int newId(int type){
      if(type == 0){
        if(arrivalQueueList.isEmpty()){
          return 1;
        } else{
          int id = arrivalQueueList.getRear().getId();
          id++;
          return id;
        }
      } else if (type == 1){
        if(departureQueueList.isEmpty()){
          return 1;
        } else{
          int id = departureQueueList.getRear().getId();
          id++;
          return id;
        }
      } else{
        return 0;
      }
    }

    void registerPassenger(int type){
      // Type: 0 = Arrival, Type:1 = Departure
      string name = "";
      string flightNumber = "";
      int priority = 0;
      int id = 0;

      if(type == 0){
        cout << "=============== Register Arrival Flights =============" <<
endl;
        cout << "Passenger Name: ";
        getline(cin, name);
        cout << "Flight Number: ";
        getline(cin, flightNumber);
        cout << "Priority (1 - Regular, 2 - Business, 3 - VIP): ";
        cin >> priority;

        id = newId(0);
        Passenger *p = new Passenger(id, name, flightNumber, priority);

        arrivalQueueList.enqueue(p);
        cout << arrivalQueueList.getRear().getName() << " had been added to
arrival queue list."  << endl;
```

```
      } else if(type == 1){
        cout << "============= Register Departure Flights ============" <<
endl;
        cout << "Passenger Name: ";
        getline(cin, name);
        cout << "Flight Number: ";
        getline(cin, flightNumber);
        cout << "Priority (1 - Regular, 2 - Business, 3 - VIP): ";
        cin >> priority;

        id = newId(1);
        Passenger *p = new Passenger(id, name, flightNumber, priority);

        departureQueueList.enqueue(p);
        cout << departureQueueList.getRear().getName() << " had been added to
departure queue list."  << endl;
      }
      cout << "=========================================================" <<
endl << endl;
    }

    void removePassenger(int type){
      cout << "============= Remove Passenger from Queue ===========" << endl;
      if(type == 0){
        cout << arrivalQueueList.getFront().getName() << " had been remove
from arrival queue list."  << endl;
        arrivalQueueList.dequeue();
      } else if(type == 1){
        cout << departureQueueList.getFront().getName() << " had been remove
from departure queue list."  << endl;
        departureQueueList.dequeue();
      }
      cout << "=========================================================" <<
endl << endl;
    }

    void viewFirstPassenger(int type){
      cout << "=========== View First Passenger in Queue ==========" << endl;
      if(type == 0){
        arrivalQueueList.getFront().DisplayPassengerInfo();
      } else if(type == 1){
        departureQueueList.getFront().DisplayPassengerInfo();
      }
      cout << "=========================================================" <<
endl << endl;
    }
```

```
    void viewLastPassenger(int type){
      cout << "=========== View Last Passenger in Queue ==========" << endl;
      if(type == 0){
        arrivalQueueList.getRear().DisplayPassengerInfo();
      } else if(type == 1){
        departureQueueList.getRear().DisplayPassengerInfo();
      }
      cout << "=========================================================" <<
endl << endl;
    }

    void viewAllPassenger(int type){
      cout << "=========== View All Passengers in Queue ==========" << endl;
      if(type == 0){
        arrivalQueueList.displayQueue();
      } else if(type == 1){
        departureQueueList.displayQueue();
      }
      cout << "=========================================================" <<
endl << endl;
    }

    void queueStatus(int type){
      if (type == 0) {
          cout << "=========== Arrival Queue Status ==========" << endl;
          if (arrivalQueueList.isEmpty()) {
              cout << "The arrival queue is currently empty." << endl;
          } else if (arrivalQueueList.isFull()) {
              cout << "The arrival queue is currently full." << endl;
          } else {
              cout << "The arrival queue has available space." << endl;
          }
      } else if(type == 1){
        cout << "========= Departure Queue Status ==========" << endl;
          if (departureQueueList.isEmpty()) {
              cout << "The departure queue is currently empty." << endl;
          } else if (departureQueueList.isFull()) {
              cout << "The departure queue is currently full." << endl;
          } else {
              cout << "The departure queue has available space." << endl;
          }
      }
      cout << "=========================================================" <<
endl << endl;
    }
};
```

```
int main(){
  Simulation s1;
  s1.runSimulation();

  return 0;
}
```

4. Project Execution and Management

4.1 Development Activities

Our team began with a brainstorming session with clear focus of the project itself, aims and goals and roles and responsibilities of each team member. The following deliverables have been identified, system requirements, coding modules and use case diagram. These help with the coordination through different stages of the system such as the system design, coding, testing and integration in ensuring progress within the timeline.

4.2 Task Distribution

Member	Project Synopsis	Project Objectives	System Requirements	System Analysis	System Design	Coding (Passenger Class)	Coding (Queue Class)	Coding (Simulation Class)	Conclusion
Brendan Chia Yan Fei	✓		✓				✓		
Goh Jiale		✓		✓				✓	
Yasmin					✓	✓			
Salman								✓	✓

4.3 Log Sheet

Meeting No.	Date	Participants	Discussion Points	Outcome
1	01/01/2025	All Members	Discussed project goals and assigned individual tasks.	Finalized project plan and responsibilities.

2	05/01/2025	All Members	Reviewed system requirements and diagrams.	Approved UML diagrams and confirmed coding approach.
3	10/01/2025	Yasmin, Goh, Brendan	Reviewed implementation of the Queue and Passenger classes.	Made adjustments for dynamic memory allocation.
4	11/01/2025	Brendan, Goh, Salman	Final integration of code. Simulation testing.	Fixed bugs in the Simulation class and verified output.
5	12/01/2025	All Members	Finalized report	Compiled the complete project report.

4.4 Conclusion

The Flight Passenger Queue Simulation at Airport project managed to prove that data structures such as queues can be applied in real world problem settings to answer them as efficiently as possible. We designed a simulation that puts passengers first while also handling arrivals and departures quickly through an object oriented application. Not only did this project help us become even more proficient in C++ language, it taught us the value of teamwork, and systematic problem solving. Collaboration and iteration solved challenges, including memory management and input validation. The project achieved its overall objectives and provided a useful learning exercise.

4.5 Demo Video Link

Youtube Link